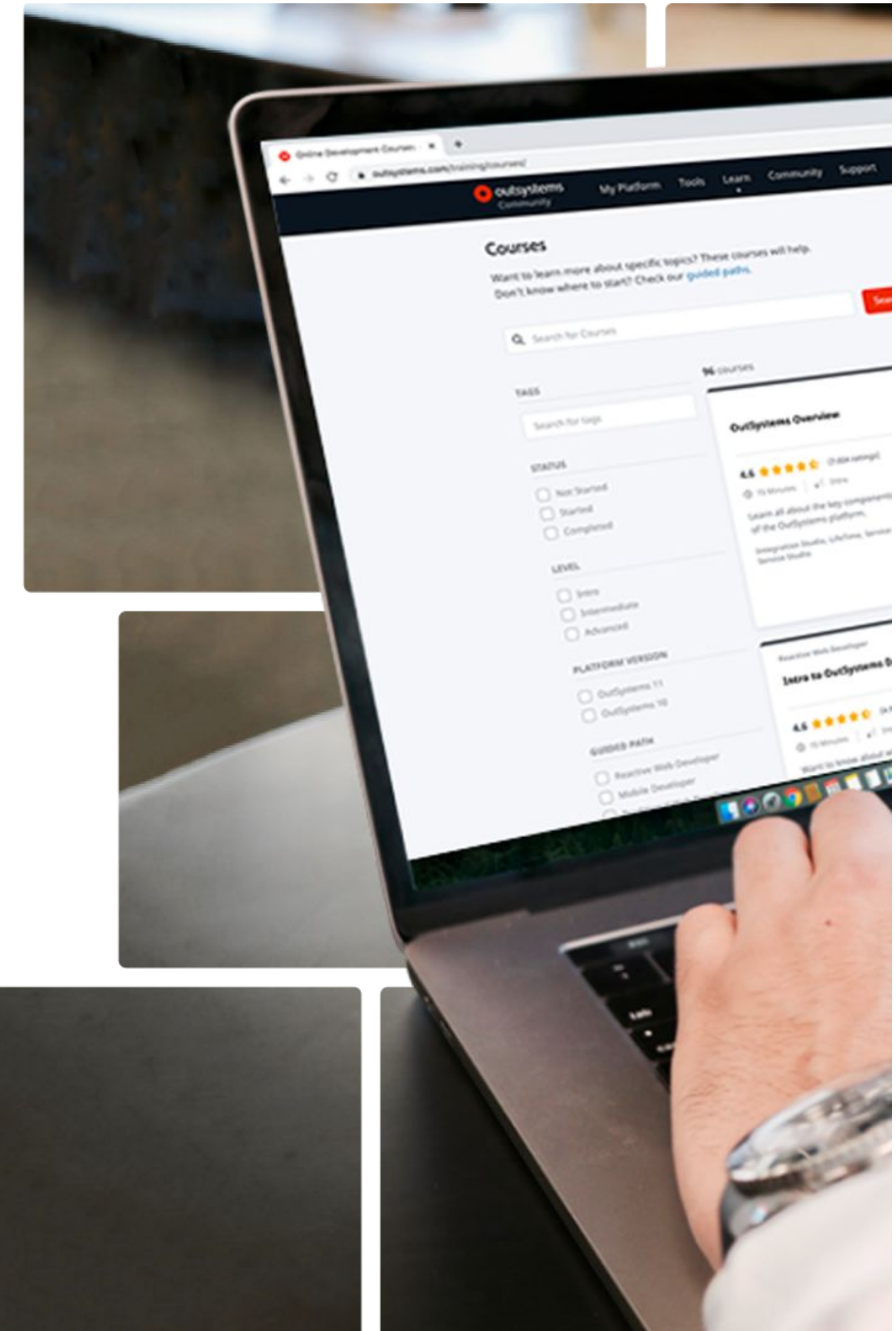


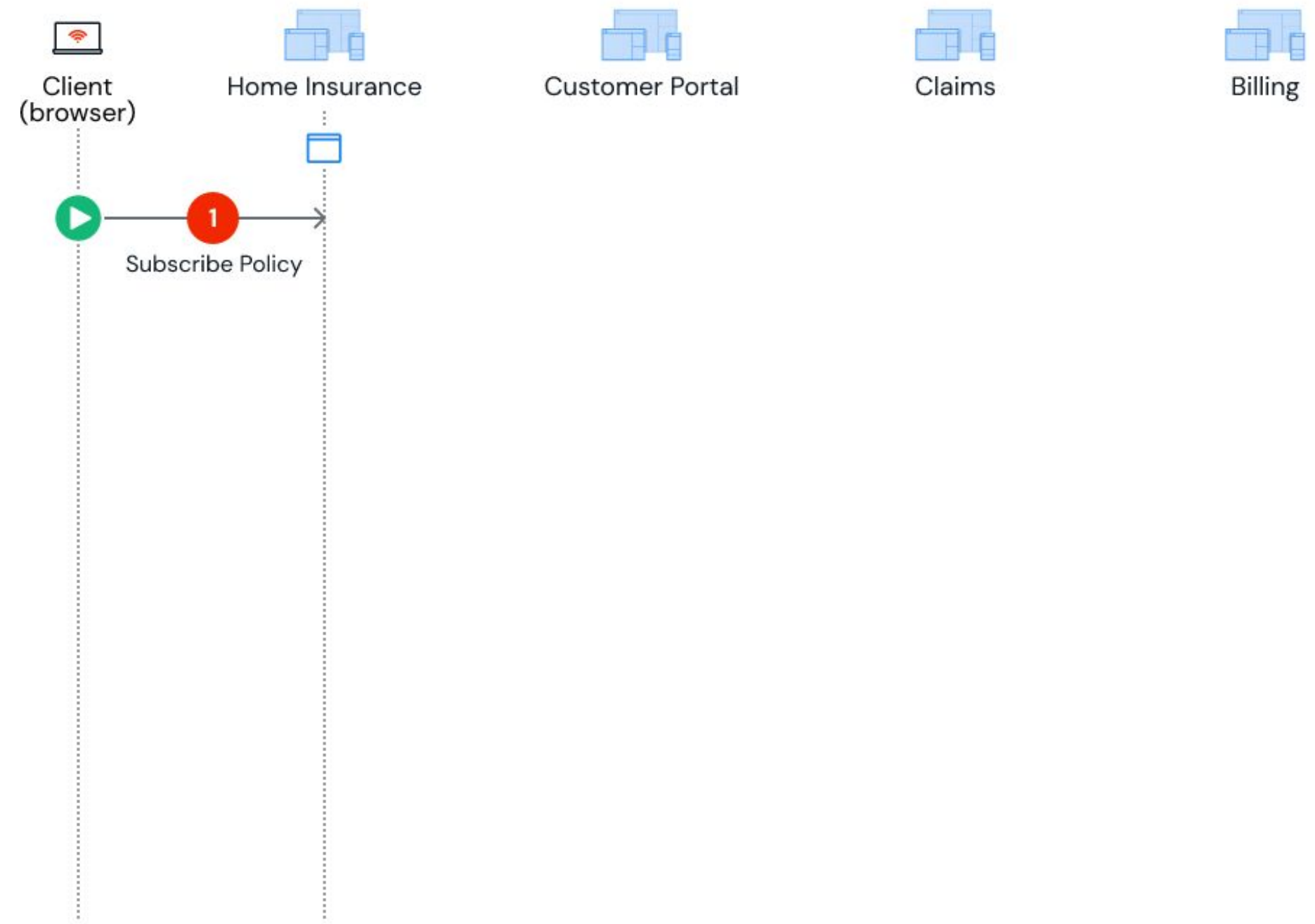
Using Service Actions

Use Case
Failure Scenarios
Compensation Mechanisms



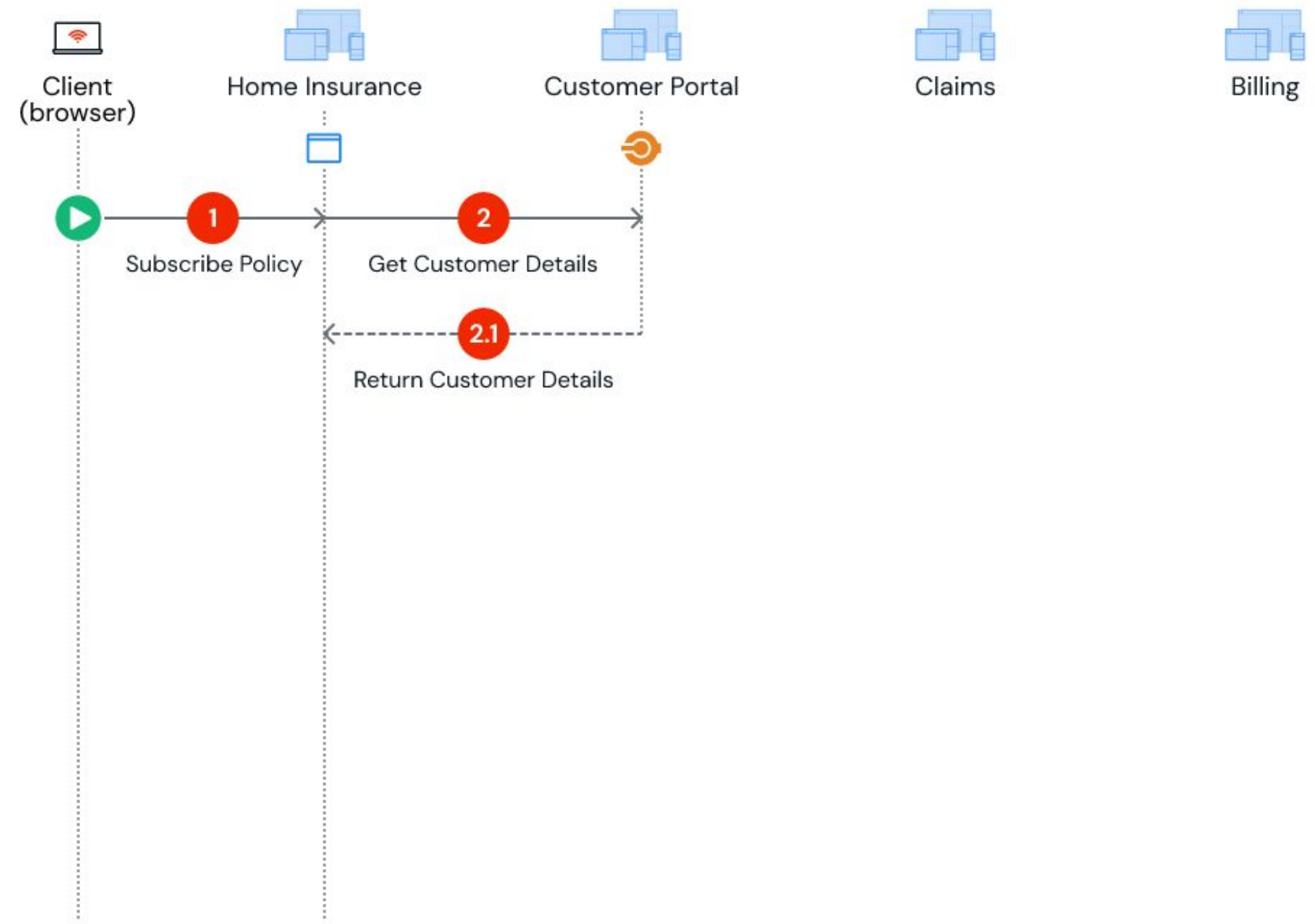
Insurance Policy Online Subscription Use Case

- Customer subscribes to insurance policy
 - Home Insurance
 - Logic is triggered and orchestrated



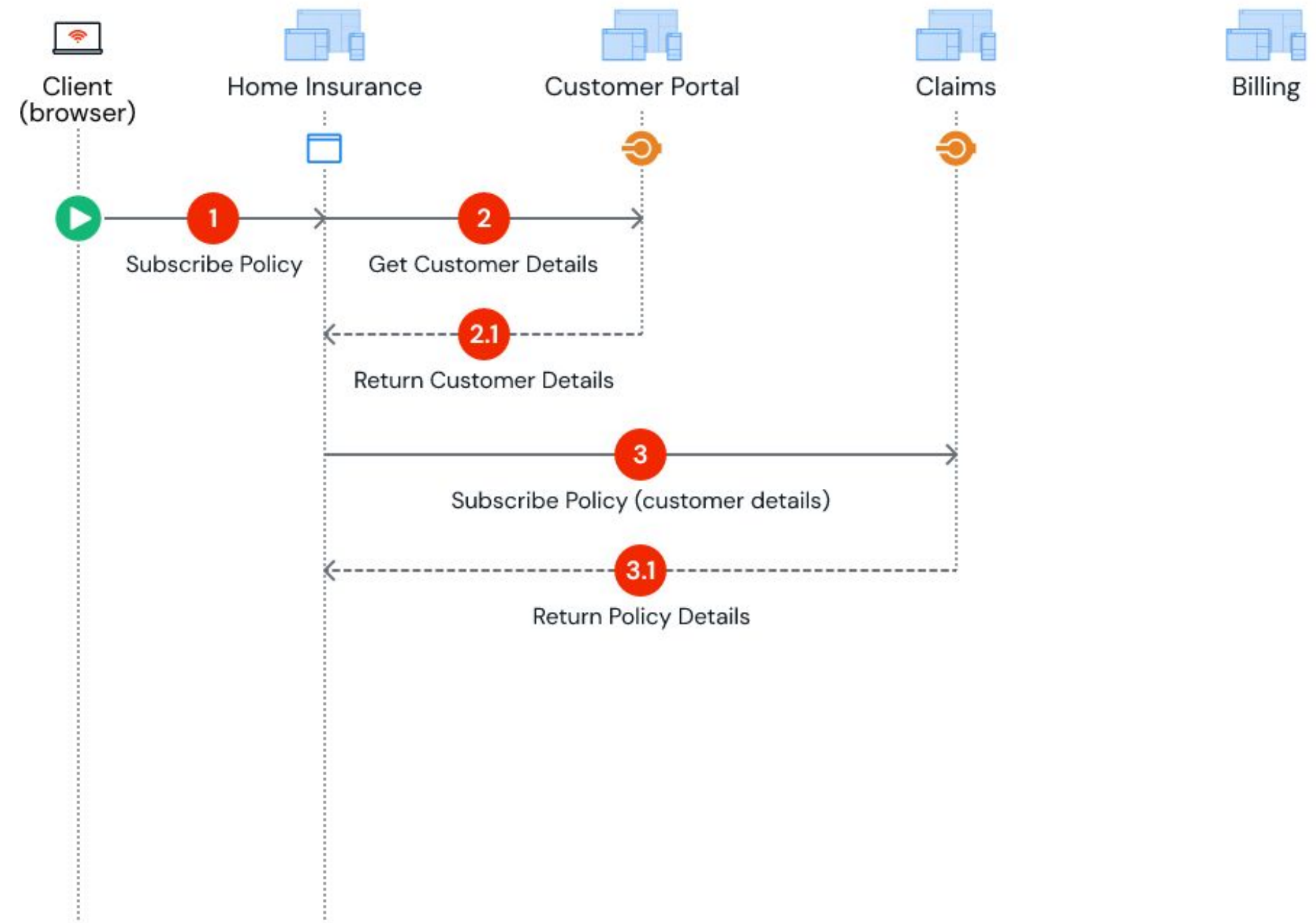
Insurance Policy Online Subscription Use Case

- Customer subscribes to insurance policy
- Get customer details
 - Customer Portal



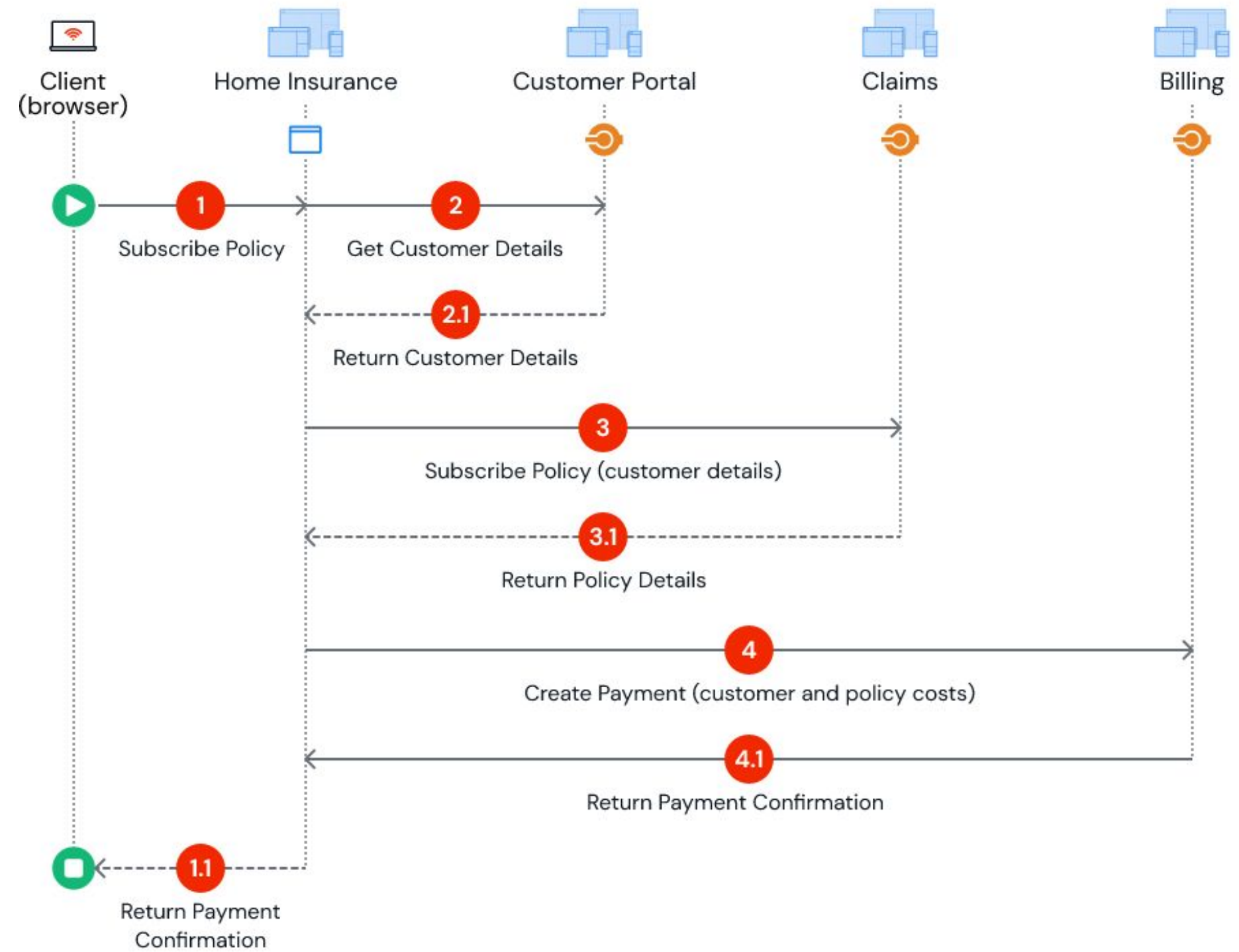
Insurance Policy Online Subscription Use Case

- Customer subscribes to insurance policy
- Get customer details
- Subscribe customer and return policy details
 - Claims

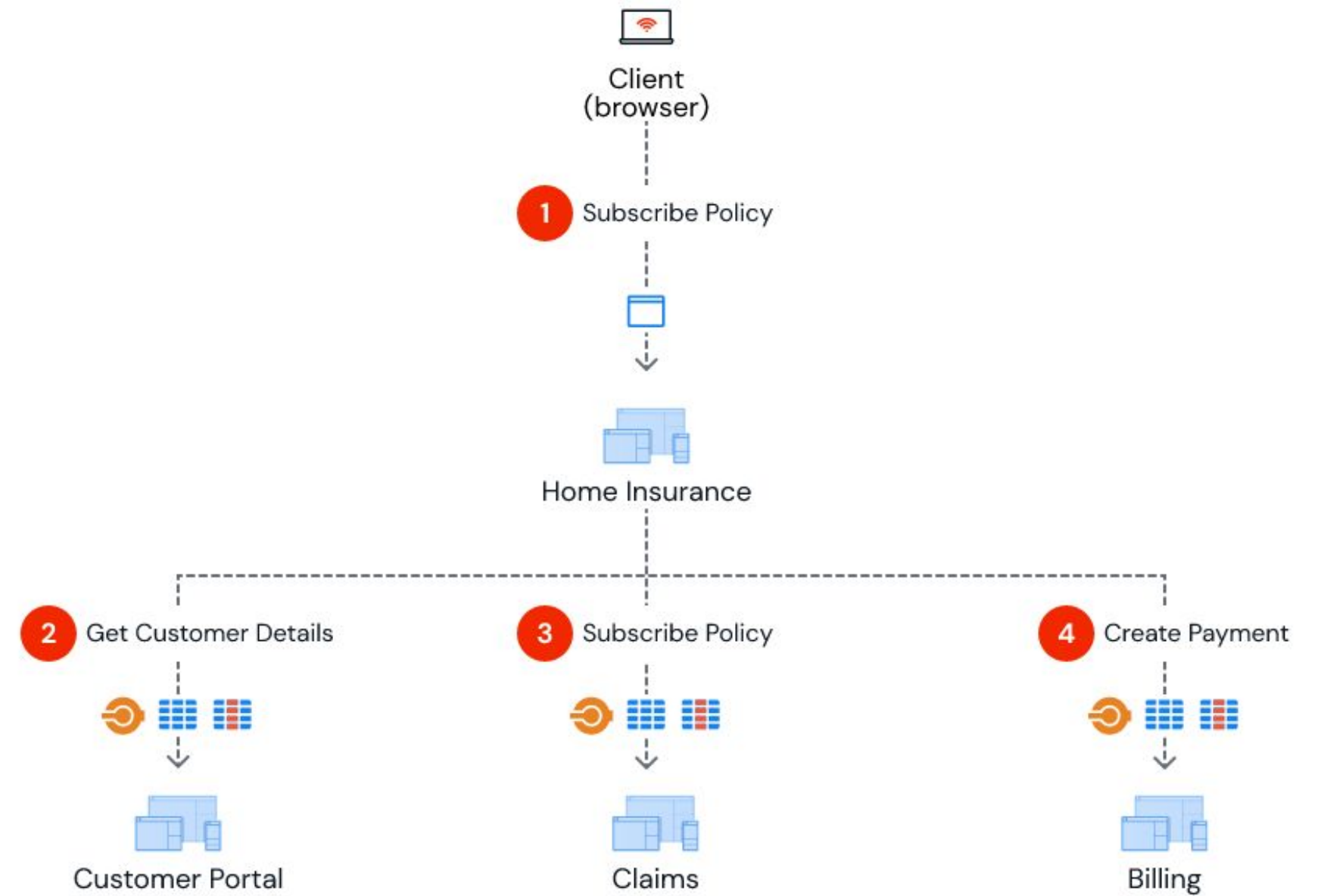


Insurance Policy Online Subscription Use Case

- Customer subscribes to insurance policy
- Get customer details
- Subscribe customer and return policy details
- Process payment and return confirmation
 - Billing

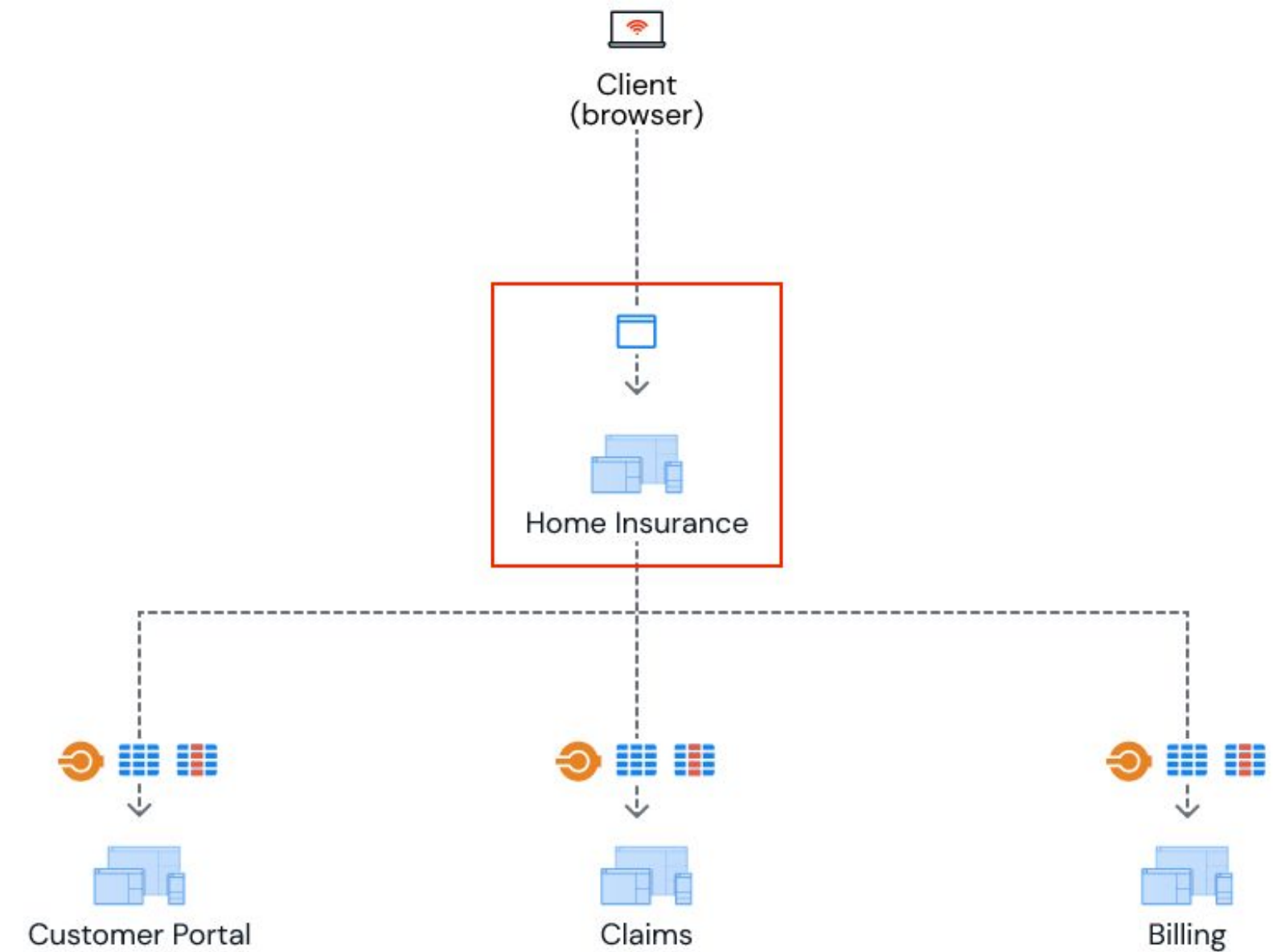


Insurance Policy Online Subscription - Architecture

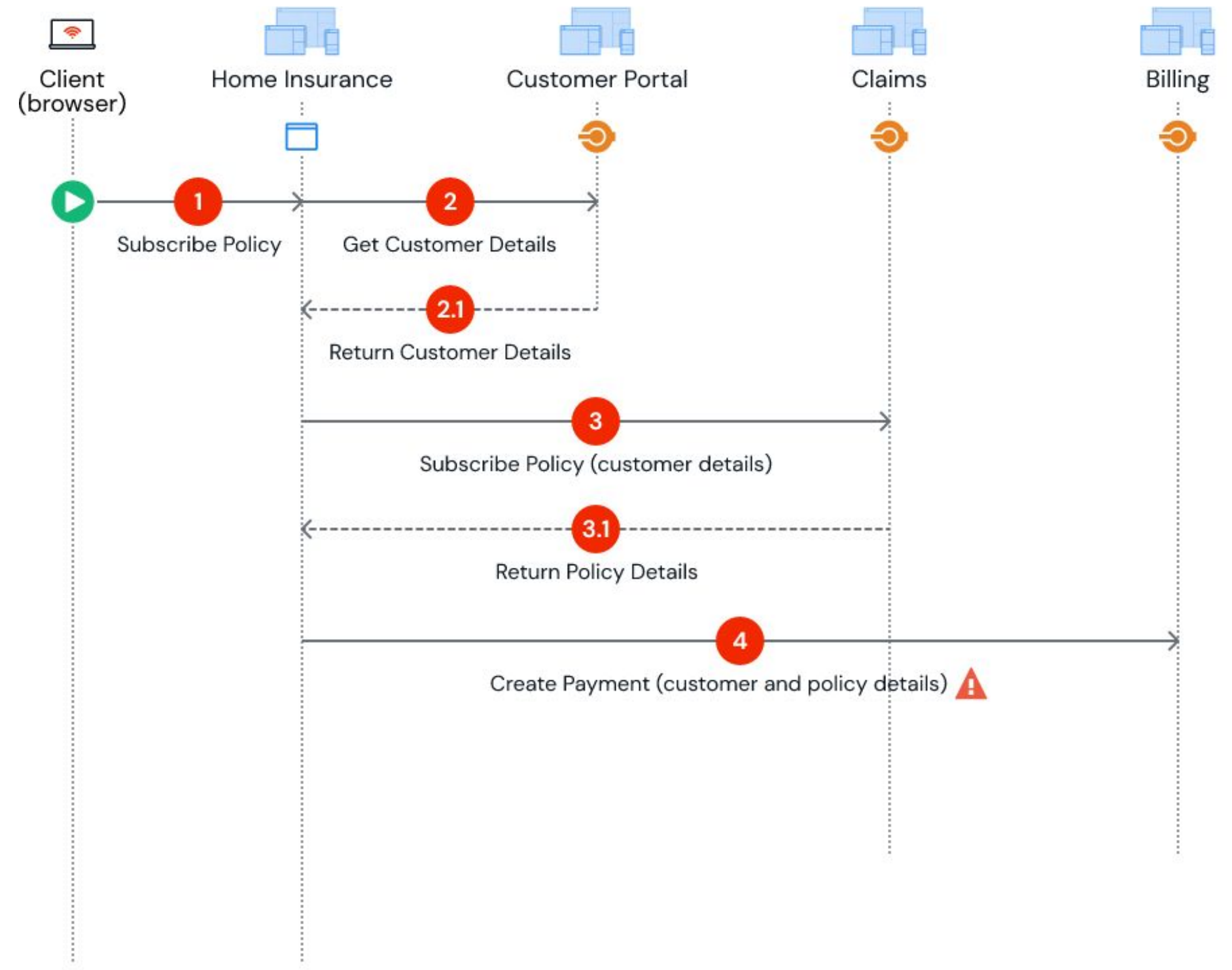


Insurance Policy Online Subscription - Architecture

- Orchestration done in Home Insurance App
- Consistent state at the end of each transaction
 - Even if calls fail

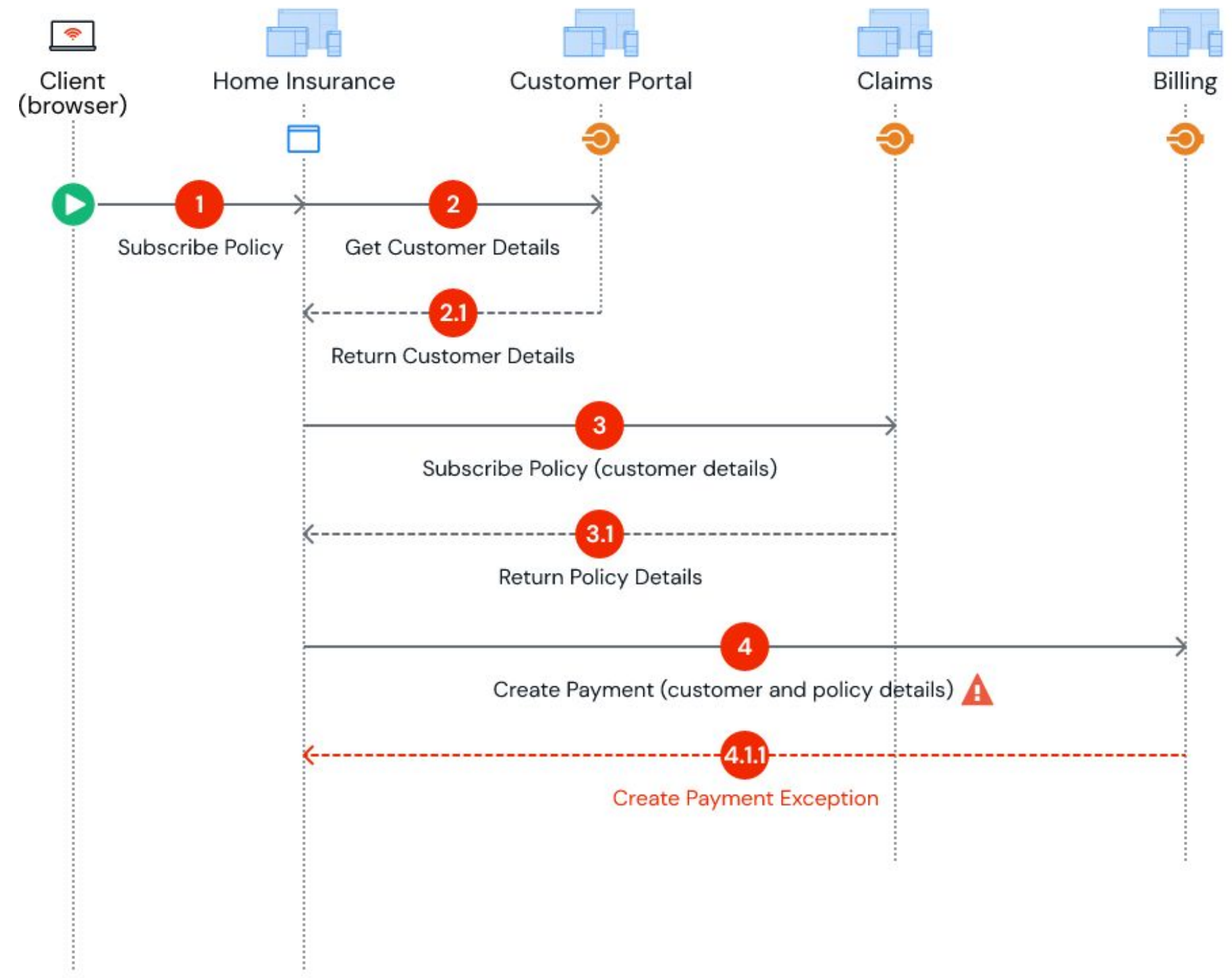


Failure Scenario



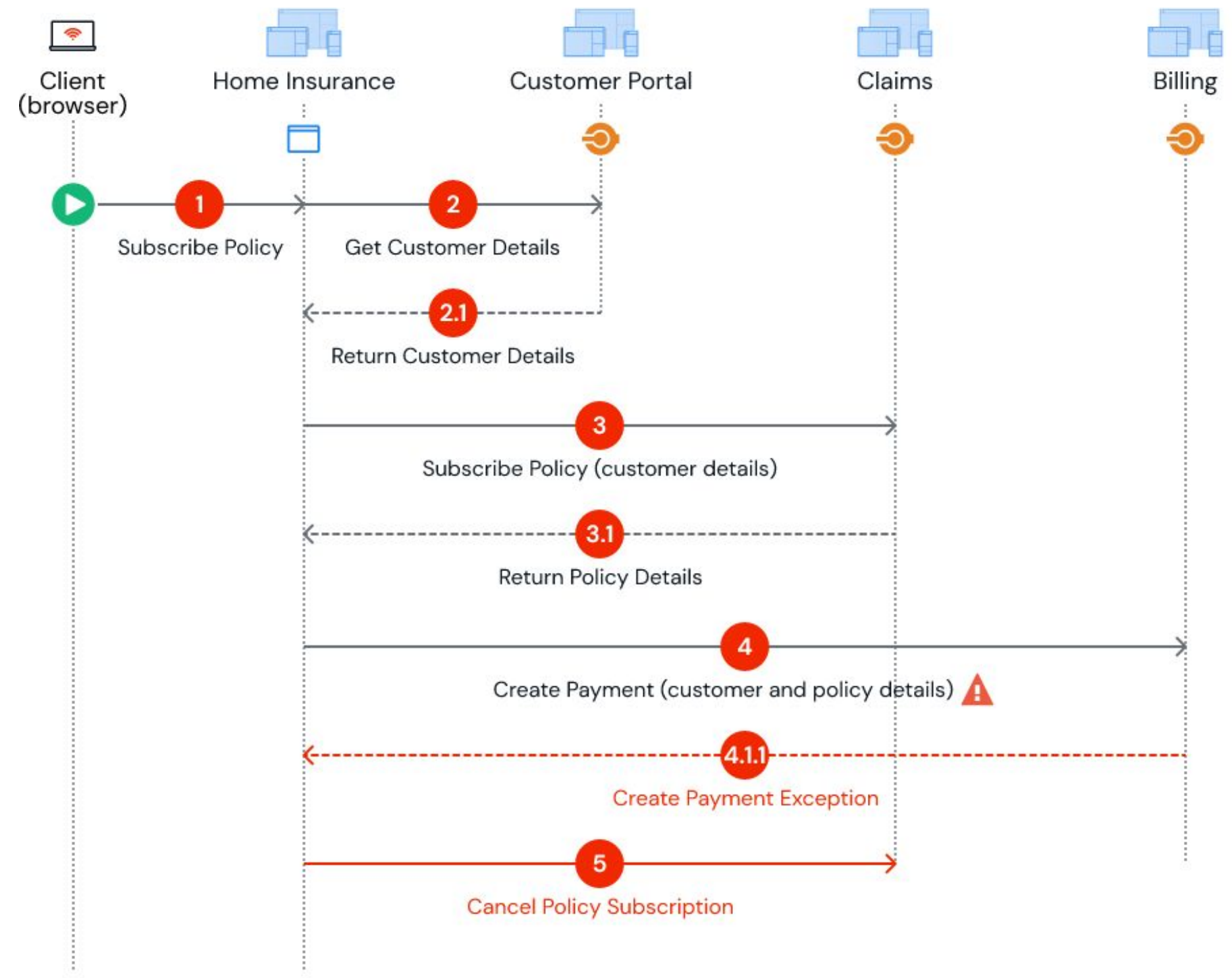
Failure Scenario

- Exception prevents payment creation



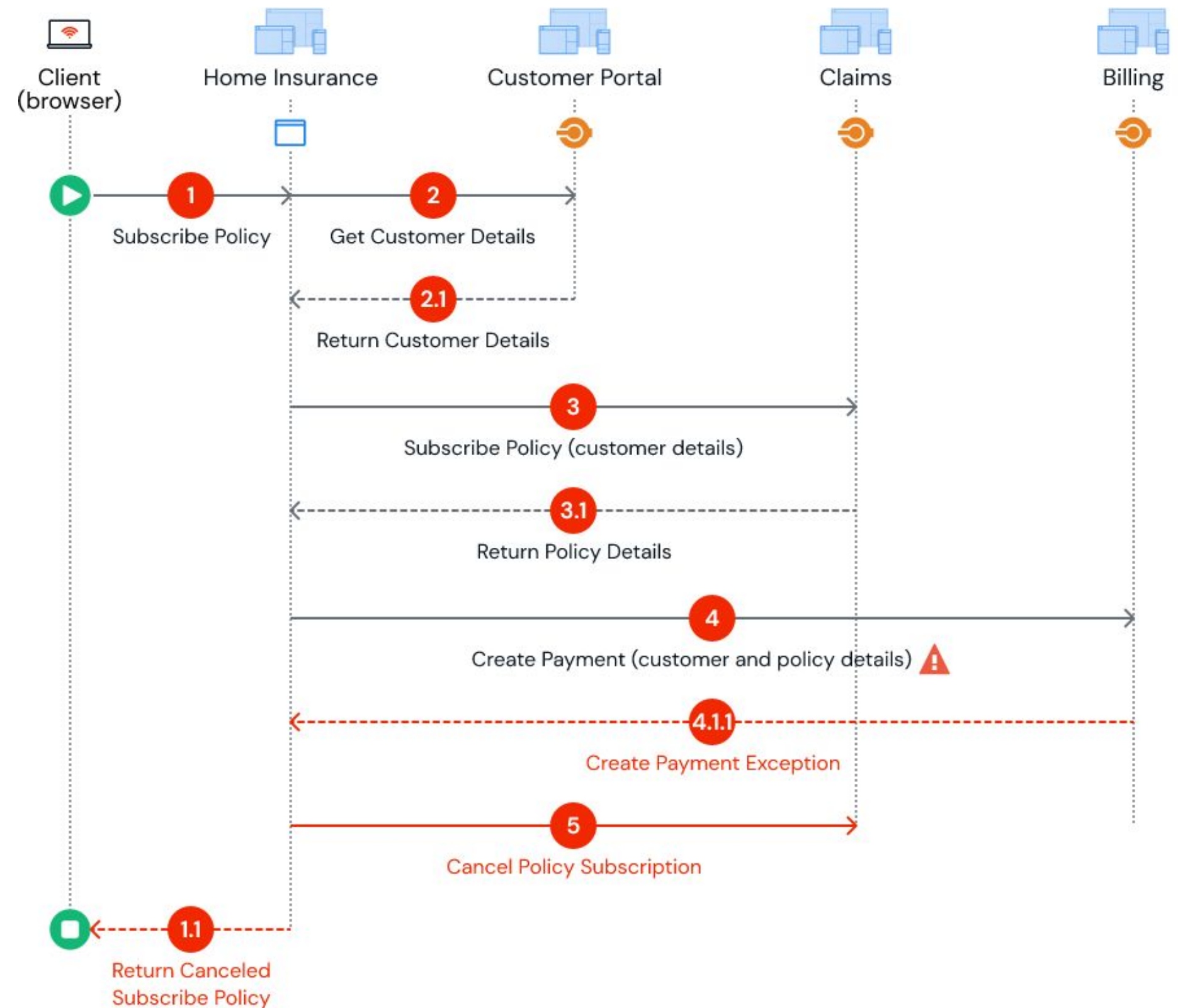
Failure Scenario

- Exception prevents payment creation
- Cancel policy subscription



Failure Scenario

- Exception prevents payment creation
- Cancel policy subscription
- Return canceled subscribe policy

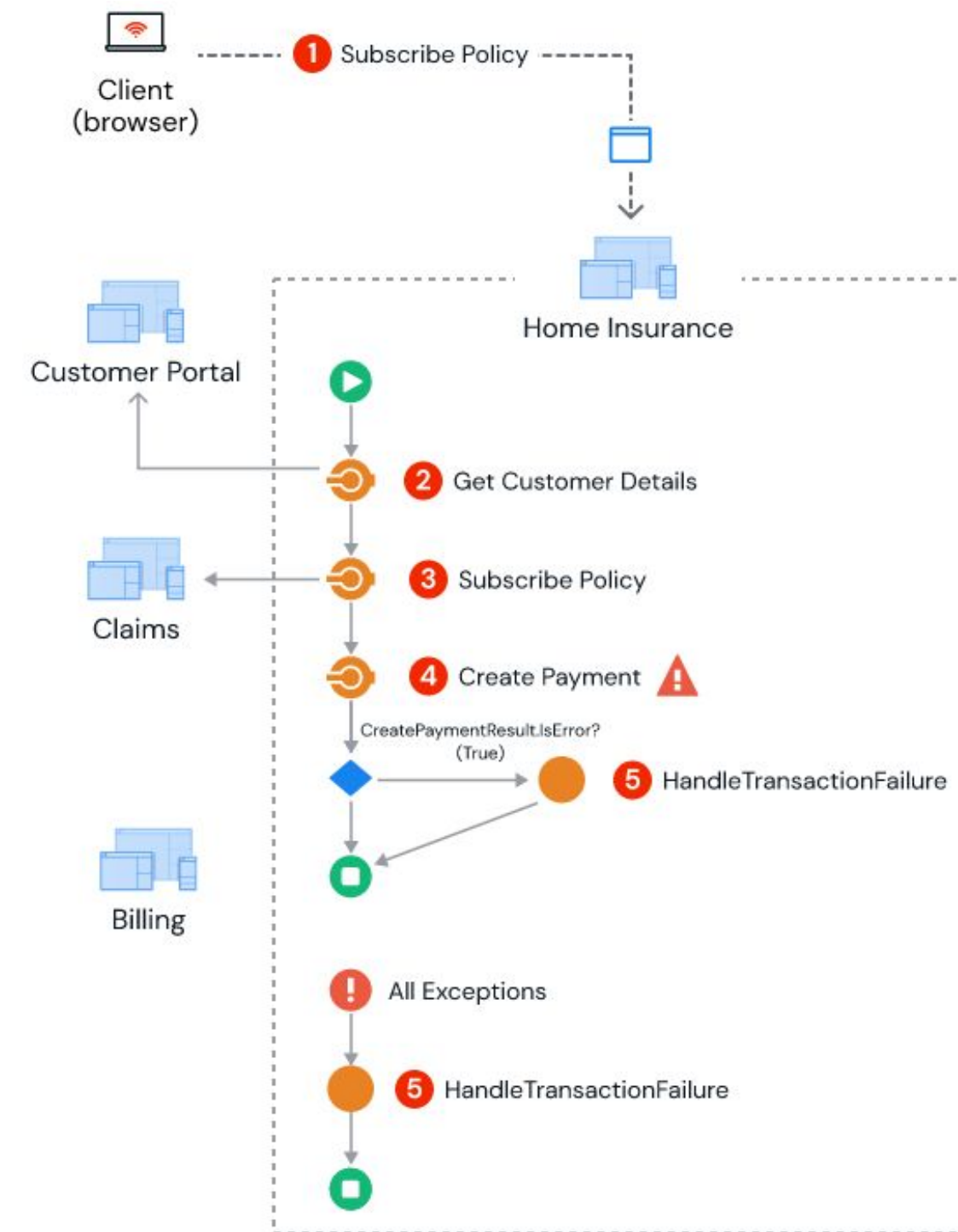


Compensation Mechanisms

Failure scenarios require compensation mechanisms to ensure **data consistency** across apps.

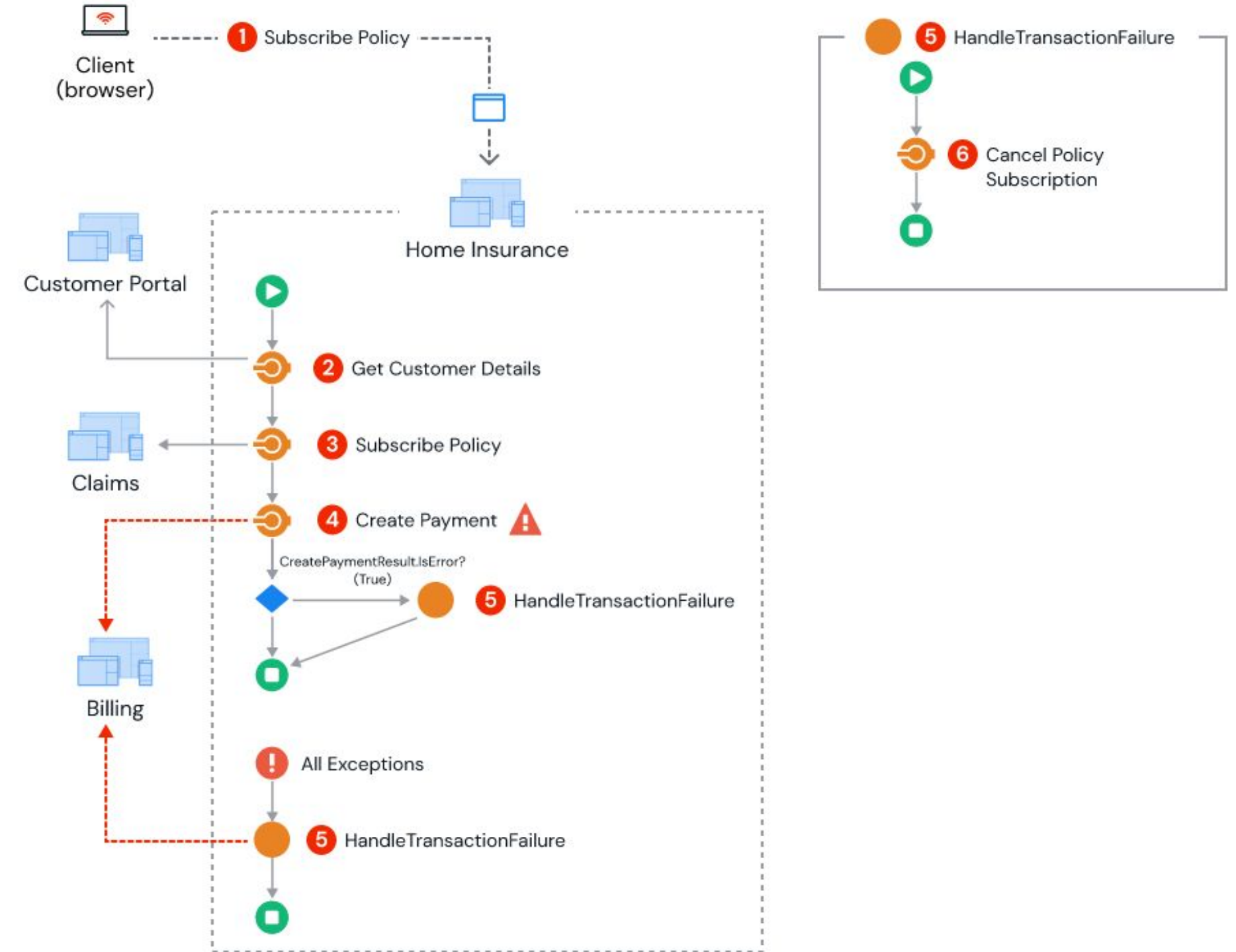
Compensation Mechanisms - Implementation

- Depends on the use case
- Revert state to a consistent one
- Done at the Exception flow



Compensation Mechanisms - Implementation

- Depends on the use case
- Revert state to a consistent one
- Done at the Exception flow



Compensation Mechanisms

- Roll back or soft delete data
 - Create reverse operation
- Execute additional logic to ensure consistency
- Cope with concurrent transactions



Poor Isolation Issues in Concurrent Transactions



Lost updates



Dirty reads



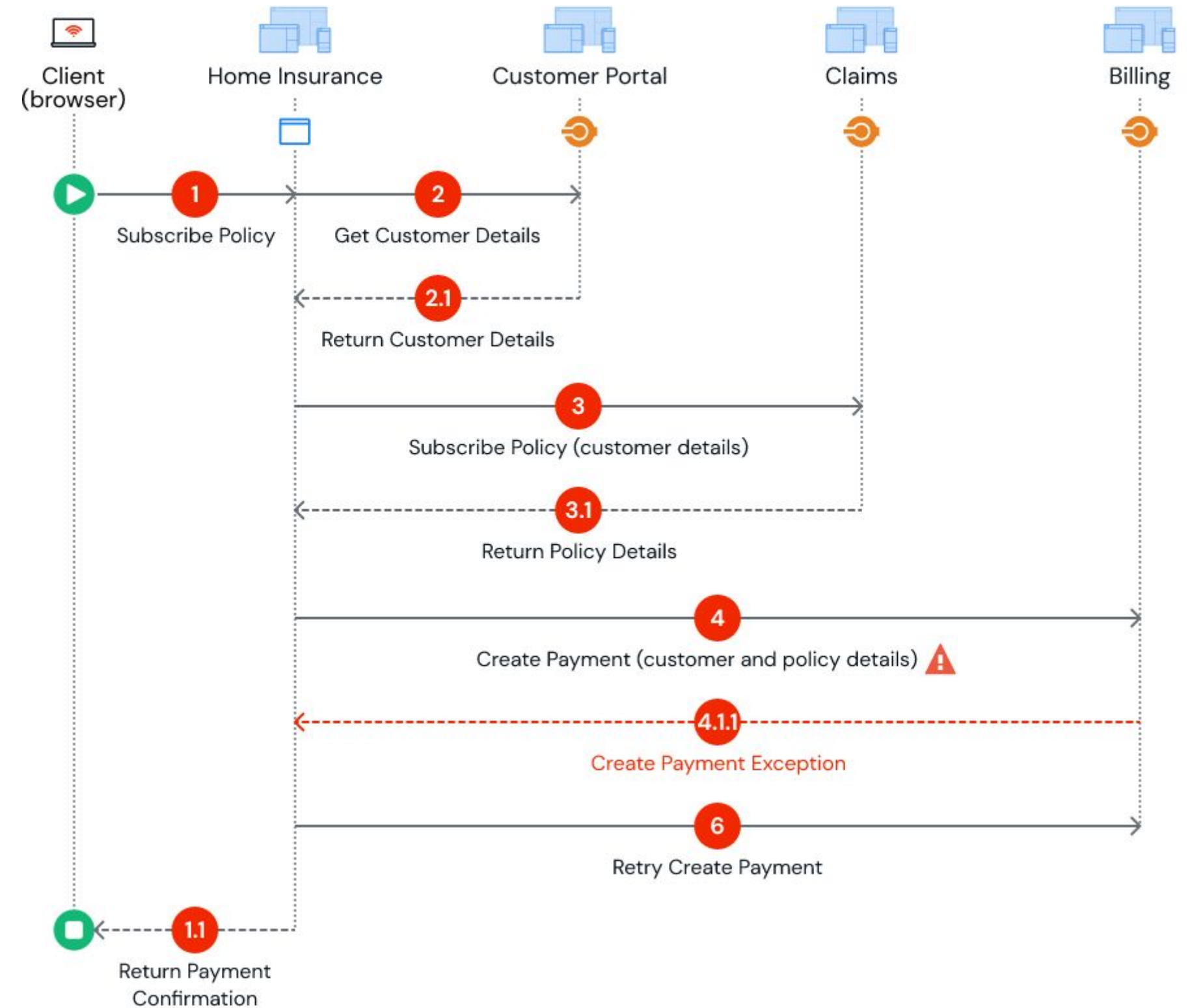
Non repeatable reads

Retry Approach

- Retry failed operations
- Immediate recovery
 - Avoids compensation

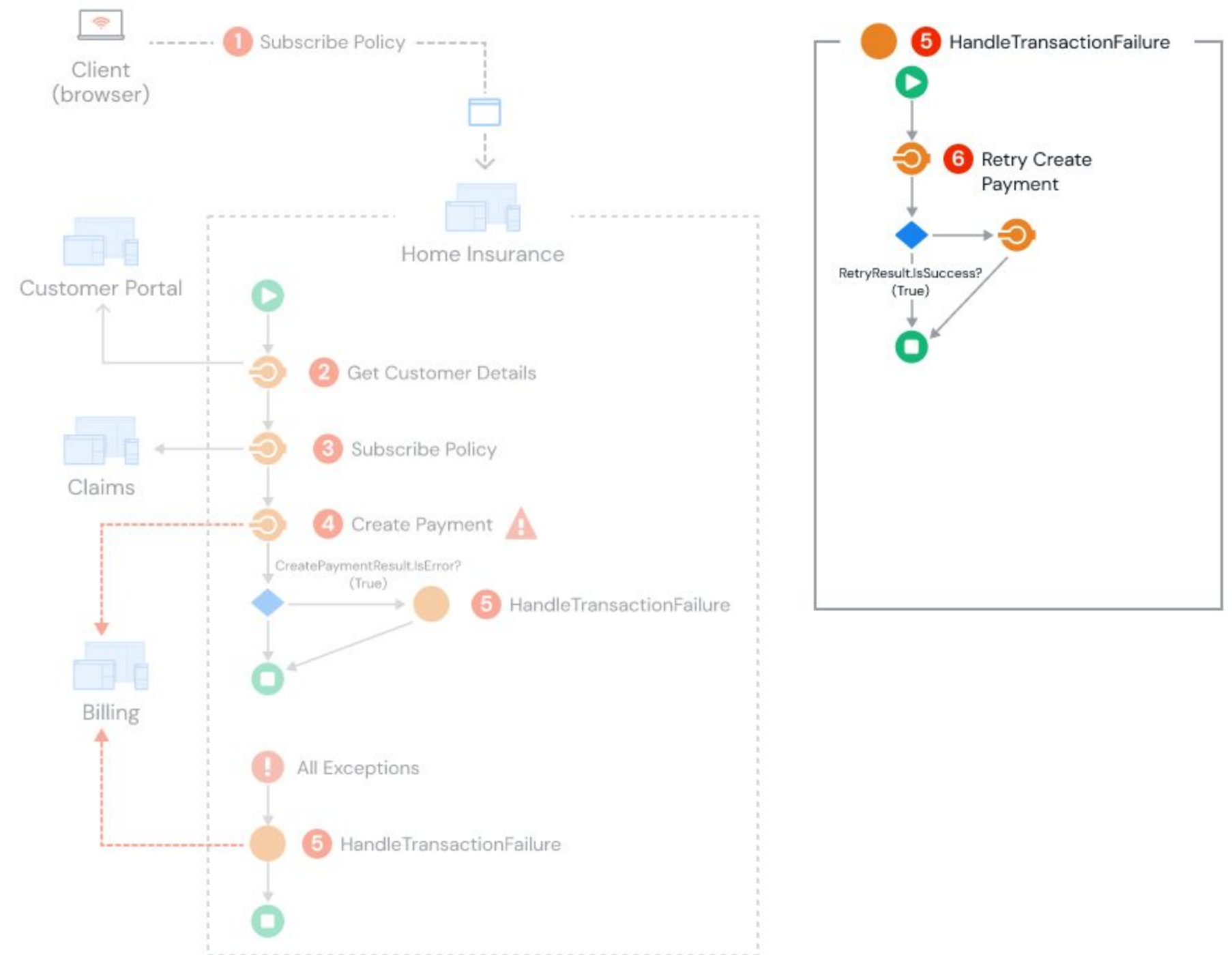
Insurance Policy Online Subscription with Retry Approach

- Creation operation is retried
 - Immediate transaction recovery
- No cancellation necessary



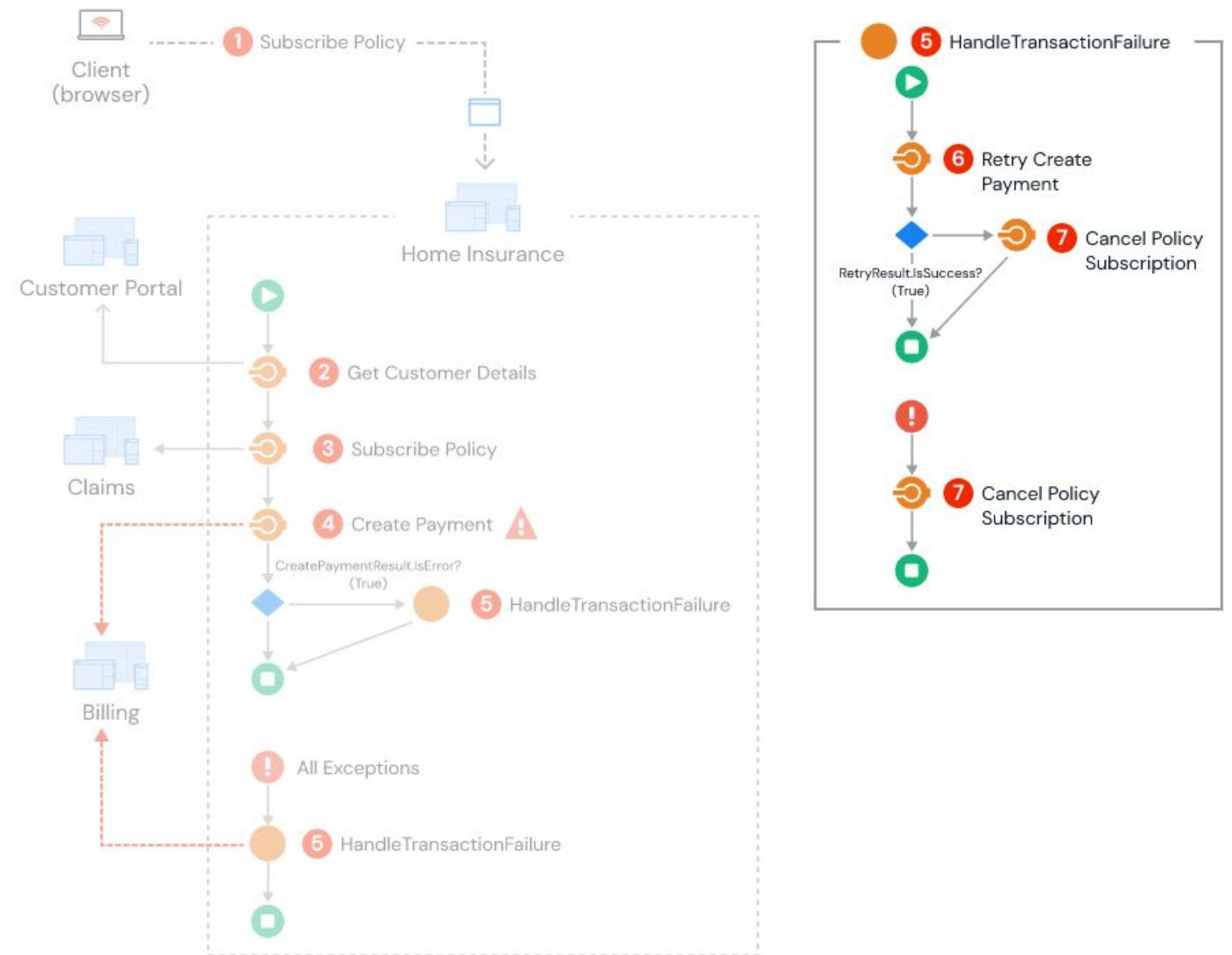
Retry - Implementation

- Retry Create Payment



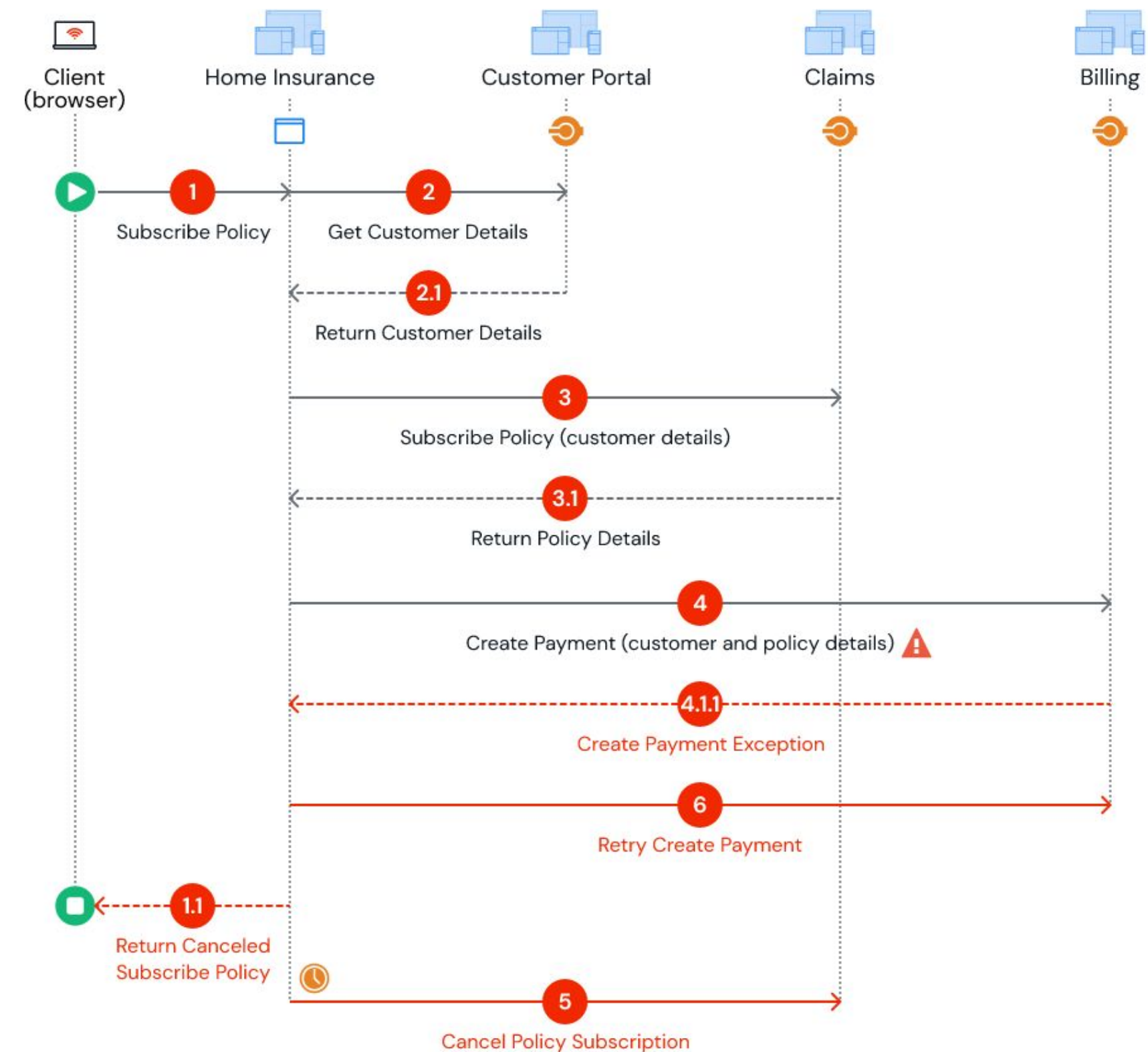
Retry - Implementation

- Retry Create Payment



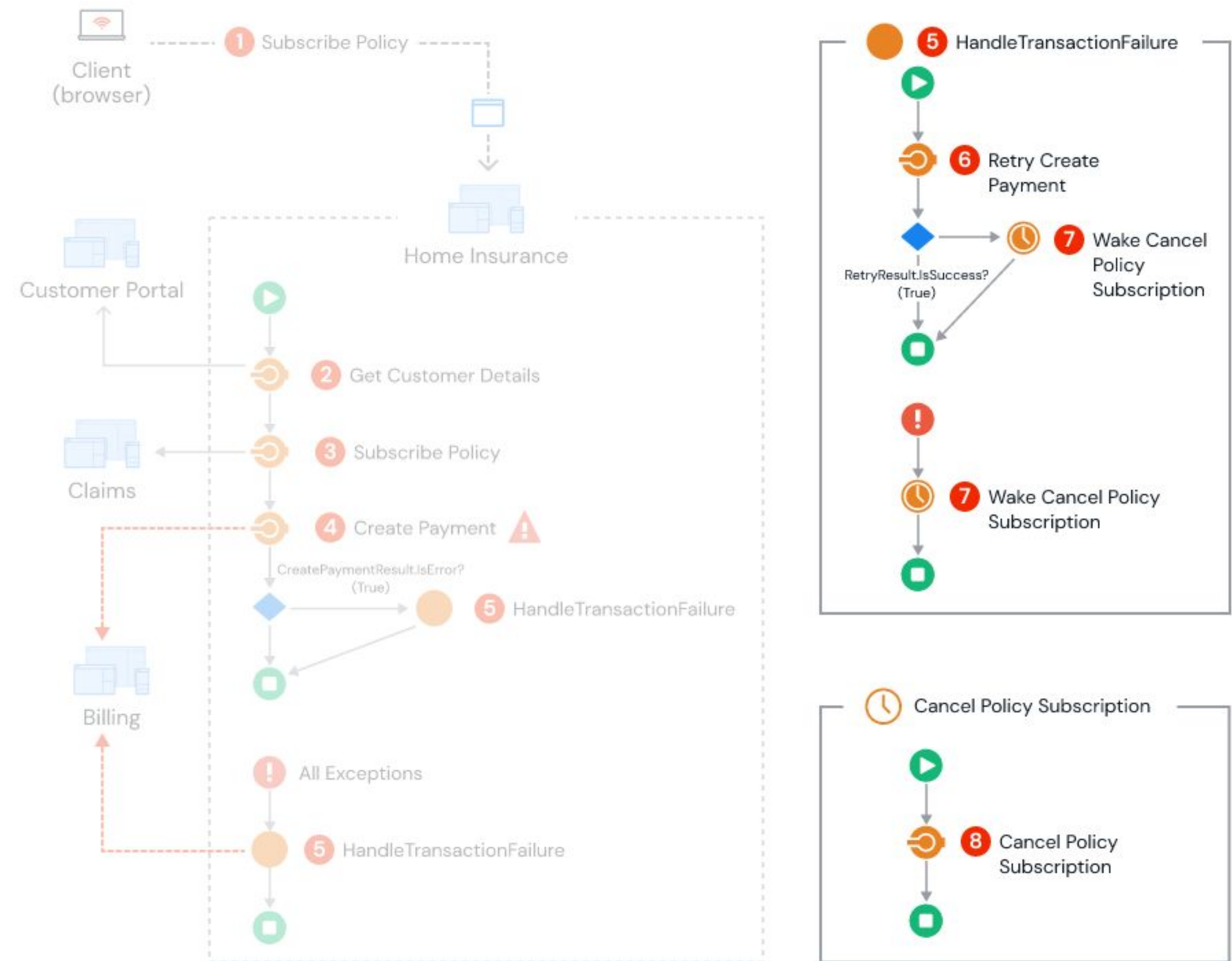
Retry Recovery and Asynchronous Compensation

- Retry does not ensure recovery
- Compensation can follow asynchronous approach



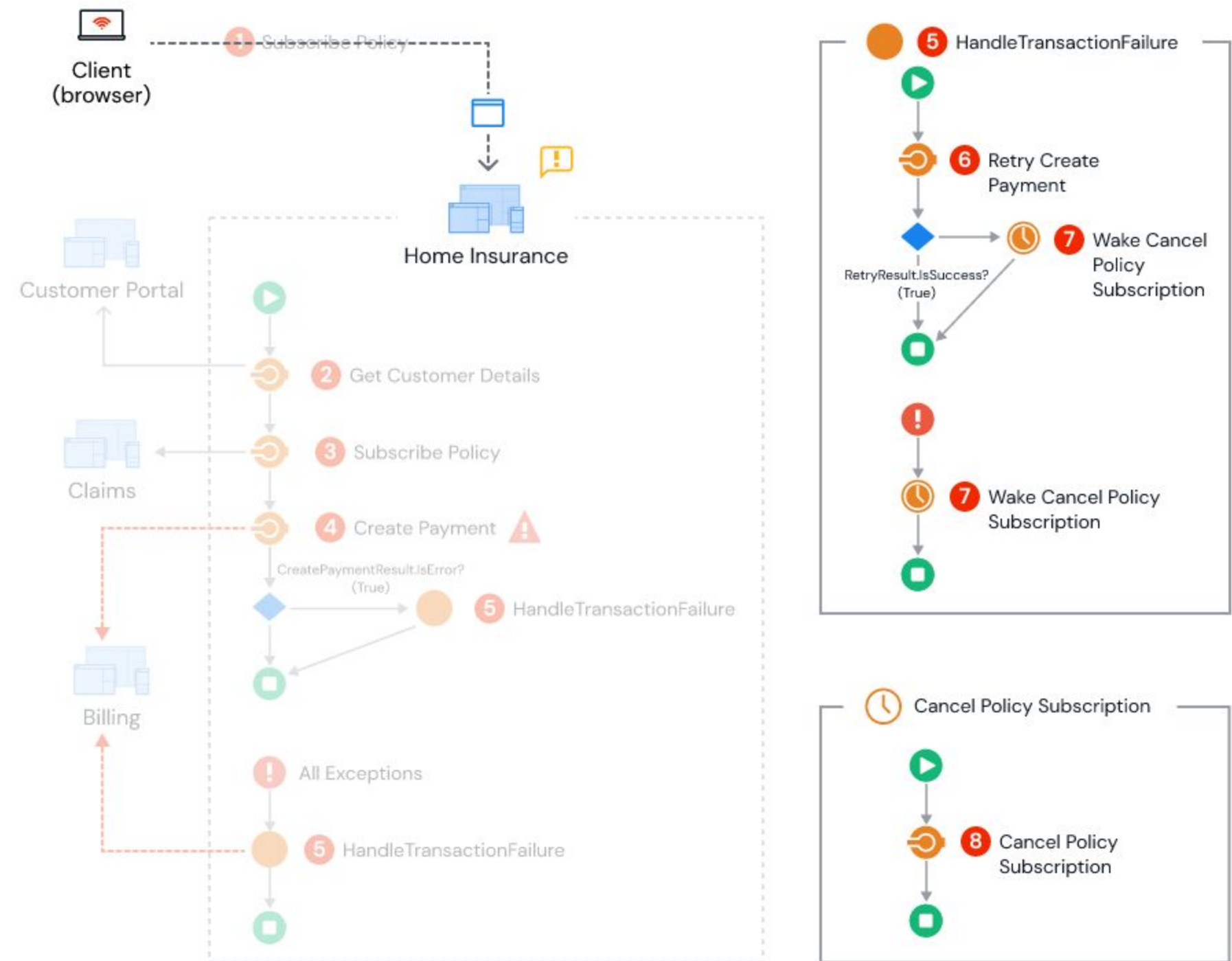
Asynchronous Compensation - Implementation

- Compensation logic executed in a Timer
- More retries if necessary



Asynchronous Compensation - Implementation

- Compensation logic executed in a Timer
- More retries if necessary
- Fallback should exist



Asynchronous Compensation - Implementation

- Compensation logic executed in a Timer
- More retries if necessary
- Fallback should exist
- Multiple runs should result the same outcomes

